

《机器学习工程师知识库》

从入门到精通 · 职业技能培训教材

版本：2026 最新版

适用人群：零基础到进阶开发者

学习目标：掌握核心算法原理、熟练运用 sklearn、具备独立建模能力

第1章 机器学习概述与职业环境搭建

1.1 机器学习核心概念与职业定位

机器学习 (Machine Learning, ML) 是人工智能的核心分支，它使计算机能够从数据中自动学习规律，而无需显式编程。对于机器学习工程师而言，核心职责包括数据清洗、特征工程、模型训练、调优与部署。

机器学习可分为三大类型：监督学习（有标签数据，如房价预测、客户流失预警）、无监督学习（无标签数据，如客户分群、异常检测）和强化学习（通过环境交互学习，如游戏AI、自动驾驶决策）。

知识点讲解

Tom Mitchell 给出的经典定义：一个计算机程序从经验E中学习，针对某类任务T和性能度量P，如果它在T上的性能（由P衡量）随着经验E的增加而提高，则称该程序从经验中学习。

在实际工作中，机器学习工程师需要理解业务问题、选择合适算法、评估模型效果，并将模型部署到生产环境。这要求工程师不仅懂算法，还要懂工程实现。

操作步骤与工作流程

步骤一：业务理解。明确问题类型（分类/回归/聚类），确定评估指标。

步骤二：数据收集与清洗。处理缺失值、异常值、重复数据。

步骤三：探索性数据分析（EDA）。使用统计方法和可视化了解数据分布。

步骤四：特征工程。构造、选择和转换特征。

步骤五：模型训练与验证。划分训练集/验证集/测试集，交叉验证。

步骤六：模型调优。网格搜索、随机搜索、贝叶斯优化。

步骤七：模型部署与监控。API封装、A/B测试、性能监控。

注意事项

数据泄漏防范

任何基于全量数据的预处理（如标准化、特征选择）必须在训练集上 fit，在测试集上仅 transform。若先用全量数据标准化再划分，测试集信息会泄漏到训练过程，导致评估结果虚高。

随机种子固定

在团队协作和结果复现时，务必设置 random_state，确保实验可复现。

常见问题

Q1：如何判断应该使用监督学习还是无监督学习？

A：如果业务场景中有明确的历史标签（如已知的违约客户名单），使用监督学习；如果仅想发现数据内在结构（如将用户自动分群），使用无监督学习。

Q2：训练集、验证集、测试集的比例如何确定？

A：数据量较小时（如几千条），常用 60/20/20 或 70/15/15；数据量较大时（百万级），测试集 5%~10% 即可。验证集用于调参，测试集仅用于最终评估。

实际案例

【案例】电商平台用户流失预警项目

某电商平台发现用户流失率上升，需要提前识别可能流失的用户并进行挽留。项目流程如下：

1. 业务定义：将30天内未登录且历史消费大于1000元定义为准流失用户，作为标签 $y=1$ 。
2. 特征构造：RFM模型特征（最近购买时间、购买频率、消费金额）、浏览行为特征、客服交互特征。
3. 数据划分：按时间顺序划分，用前6个月训练，第7个月验证，第8个月测试，避免时间泄漏。
4. 模型选择：尝试逻辑回归（可解释性强）和 XGBoost（精度高），最终采用 XGBoost 并通过 SHAP 解释特征重要性。
5. 部署上线：模型以 REST API 形式部署，每日批量预测，对高风险用户推送优惠券。上线后流失率下降 15%。

1.2 Python 环境搭建与 sklearn 快速入门

Python 是机器学习领域的事实标准语言。核心工具链包括：NumPy（数值计算）、Pandas（数据处理）、Matplotlib/Seaborn（可视化）、scikit-learn（机器学习算法库）。

知识点讲解

scikit-learn（简称 sklearn）是 Python 中最广泛使用的机器学习库，提供了统一的 API 设计：所有模型都遵循 fit（训练）→ predict（预测）→ score（评估）的接口模式。

操作步骤

步骤一：安装环境。pip install numpy pandas matplotlib scikit-learn jupyter

步骤二：导入库并加载数据集。from sklearn.datasets import load_iris

步骤三：划分数数据集。from sklearn.model_selection import train_test_split

步骤四：数据预处理。from sklearn.preprocessing import StandardScaler

步骤五：选择模型并训练。model.fit(X_train, y_train)

步骤六：评估模型。model.score(X_test, y_test)

```
# sklearn
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# 1. Load the data
iris = load_iris()
X, y = iris.data, iris.target

# 2. Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# 3. Fit the model and transform the data
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

# 4. Create the model
model = KNeighborsClassifier(n_neighbors=5)

# 5. Fit the model
model.fit(X_train_s, y_train)

# 6. Predict the results
y_pred = model.predict(X_test_s)

# 7. Print the accuracy score
print(f"Accuracy: {accuracy_score(y_test, y_pred):.4f}")
```

注意事项

stratify 参数

分类问题划分数据集时，务必使用 stratify=y

进行分层抽样，确保训练集和测试集中各类别比例与原始数据一致。这在类别不平衡时尤为重要。

版本兼容性

sklearn 不同版本的默认参数可能不同（如 SVM 的 gamma 默认值从 auto 改为 scale），生产环境需锁定版本。

常见问题

Q1：为什么测试集不能参与标准化 fit？

A：如果测试集参与 fit，测试集的信息（如均值、方差）会泄漏到训练过程，导致模型评估结果过于乐观，无法反映真实泛化能力。

Q2：random_state 的作用是什么？

A：random_state 是随机数种子，固定后每次运行代码，数据划分、模型初始化等随机过程结果一致，保证实验可复现。

实际案例

【案例】金融风控模型的环境标准化

某银行数据科学团队发现，不同成员本地运行的模型结果存在差异。经排查，原因是 sklearn 版本不一致导致默认参数不同。团队随后建立了统一的 Docker 镜像，锁定 Python 3.11 + sklearn 1.5.0 + NumPy 1.26，并在 Git 中提交 requirements.txt，确保开发、测试、生产环境完全一致。

第2章 数据预处理与特征工程

2.1 数据清洗与缺失值处理

真实世界的的数据往往存在缺失、异常、重复等问题。数据清洗是机器学习项目中最耗时的环节，通常占据整个项目 60%~80% 的时间。

知识点讲解

缺失值产生的原因包括：数据采集失败、用户未填写、系统故障等。缺失模式分为三类：完全随机缺失（MCAR）、随机缺失（MAR）和非随机缺失（MNAR）。不同模式需要不同的处理策略。

常用缺失值处理方法包括：删除法（删除含缺失值的行或列）、填充法（均值/中位数/众数填充、前向填充、模型预测填充）和插值法（线性插值、样条插值）。

操作步骤

步骤一：缺失值识别。使用 Pandas 的 `isnull().sum()` 统计每列缺失比例。

步骤二：缺失模式分析。使用 `missingno` 库可视化缺失模式，判断是否为随机缺失。

步骤三：选择处理策略。缺失比例低于 5% 可删除；5%~30% 可填充；超过 30% 考虑删除该特征或用特殊模型处理。

步骤四：实施处理。数值型用中位数填充（对异常值鲁棒）；类别型用众数或新增类别 'Unknown' 填充。

步骤五：验证效果。对比处理前后的数据分布，确保未引入偏差。

```
import pandas as pd import numpy as np from sklearn.impute import SimpleImputer
# ■■■■■ missing_ratio = df.isnull().sum() / len(df) * 100
print(missing_ratio[missing_ratio > 0]) # ■■■■■ num_imputer =
SimpleImputer(strategy='median') df['age'] =
num_imputer.fit_transform(df[['age']]) # ■■■■■ cat_imputer =
SimpleImputer(strategy='most_frequent') df['category'] =
cat_imputer.fit_transform(df[['category']]) # ■■■ Unknown ■■
df['category'].fillna('Unknown', inplace=True)
```

注意事项

避免数据泄漏

填充统计量（如均值、中位数）必须仅在训练集上计算，然后应用到测试集。若用全量数据计算，会导致信息泄漏。

区分缺失含义

有时缺失本身具有业务含义（如用户未填写收入可能表示无收入）。此时不应简单填充，而应保留缺失标记或单独分析。

常见问题

Q1：为什么中位数填充比均值填充更鲁棒？

A：中位数不受极端异常值影响。若数据中存在极大异常值，均值会被拉偏，而中位数保持稳定。

Q2：缺失比例超过多少应该删除整列？

A：一般经验是缺失比例超过 30%~50% 时考虑删除该列。但需结合业务判断，若该特征极其重要，即使缺失多也应保留并采用高级填充策略。

实际案例

【案例】医疗数据缺失值处理

某医院糖尿病预测项目中，血糖记录缺失率达 25%。经分析发现，缺失主要集中在门诊患者（住院患者记录完整）。团队没有直接填充，而是构造了新特征 `is_glucose_missing` 作为标识，并对门诊和住院患者分别建立子模型，最终 AUC 提升 0.03。

2.2 特征缩放与编码技术

机器学习算法对特征的尺度和类型有严格要求。数值特征需要缩放（标准化/归一化），类别特征需要编码（独热编码/标签编码/目标编码）。

知识点讲解

标准化（Standardization）将数据转换为均值为0、标准差为1的分布，公式为 $z = (x - \mu) / \sigma$ 。适用于正态分布或近似正态分布的数据，对异常值有一定容忍度。

归一化（Normalization）将数据缩放到 $[0, 1]$ 区间，公式为 $x' = (x - \min) / (\max - \min)$ 。适用于有明确边界的数据，但对异常值敏感。

类别编码方法：独热编码（One-Hot）适合无序类别，标签编码（Label Encoding）适合有序类别，目标编码（Target Encoding）适合高基数类别特征。

操作步骤

步骤一：判断数值特征分布。使用直方图或 Q-Q 图判断是否近似正态分布。

步骤二：选择缩放方法。正态分布用 `StandardScaler`；非正态或有界数据用 `MinMaxScaler`；含异常值用 `RobustScaler`。

步骤三：类别特征分析。统计类别数量和频率，判断使用何种编码。

步骤四：实施编码。低基数 (<10) 用 OneHotEncoder；高基数 (>100) 用 TargetEncoder 或 Embedding。

步骤五：组合特征。对数值特征进行多项式扩展或交互特征构造。

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer # 预处理器 =
ColumnTransformer([ ('num', StandardScaler(), ['age', 'income', 'score']),
('cat', OneHotEncoder(drop='first', sparse_output=False), ['gender', 'city'])
]) # 拟合并转换 X_train_processed =
preprocessor.fit_transform(X_train) X_test_processed =
preprocessor.transform(X_test) # 特征名称 =
(preprocessor.named_transformers_['num'].feature_names_in_.tolist() +
preprocessor.named_transformers_['cat'].get_feature_names_out().tolist())
```

注意事项

虚拟变量陷阱

One-Hot 编码时，若将 n 个类别的特征转换为 n 列，会导致多重共线性。应设置 `drop='first'` 保留 $n-1$ 列，或后续使用带正则化的模型。

目标编码泄漏

Target Encoding 用目标变量的均值替换类别值，若直接用全量数据计算，会导致严重泄漏。应使用交叉验证方式计算（如 Category Encoders 库的 TargetEncoder）。

常见问题

Q1：决策树需要特征缩放吗？

A：不需要。决策树基于特征的排序进行分裂，不受特征绝对数值影响。但基于距离的算法（KNN、SVM、神经网络）必须进行缩放。

Q2：One-Hot 编码后特征数量爆炸怎么办？

A：对于高基数类别特征（如用户ID、IP地址），One-Hot 会产生数万列。解决方案包括：目标编码、特征哈希（Hashing）、聚类降维、或使用树模型直接处理类别特征。

实际案例

【案例】信用卡风控中的特征工程

某银行信用卡反欺诈项目中，原始特征仅 30 个。团队通过以下特征工程将特征扩展至 200+ 个：

1. 时间特征：交易小时、是否周末、距上次交易天数。
2. 聚合特征：过去1小时/1天/7天的交易金额均值、最大值、次数。
3. 交叉特征：商户类别 × 交易金额分箱、交易地点 × 时间段的欺诈率。
4. 编码处理：对商户ID（5000+类别）使用目标编码，对交易类型（10类）使用 One-Hot。

特征工程后，XGBoost 模型的 AUC 从 0.82 提升至 0.91。

第3章 线性模型：线性回归与正则化

3.1 线性回归原理与 sklearn 实现

线性回归是监督学习中最基础的回归算法，假设目标变量 y 与特征 X 之间存在线性关系。模型形式为 $\hat{y} = \beta_0 + \beta_1 x_1 + \dots + \beta_n x_n$ 。

知识点讲解

线性回归通过最小化均方误差 (MSE) 来估计参数。求解方法有两种：最小二乘法 (OLS) 直接求解正规方程，适用于特征数少、数据量不大的场景；梯度下降法迭代更新参数，适用于大规模数据和在线学习。

评估指标包括：MSE (均方误差，对异常值敏感)、RMSE (均方根误差，与目标同量纲)、MAE (平均绝对误差，对异常值鲁棒)、 R^2 (决定系数，解释方差比例)。

操作步骤

步骤一：数据探索。绘制特征与目标的散点图，初步判断线性关系。

步骤二：特征缩放。线性回归对尺度敏感，使用 StandardScaler 标准化。

步骤三：训练模型。使用 LinearRegression 拟合数据。

步骤四：评估模型。计算 R^2 、RMSE、MAE，绘制残差图检查假设。

步骤五：交叉验证。使用 cross_val_score 评估模型稳定性。

```
from sklearn.linear_model import LinearRegression from sklearn.metrics import
mean_squared_error, r2_score from sklearn.model_selection import
cross_val_score import numpy as np # 1. 训练模型 lr = LinearRegression()
lr.fit(X_train_scaled, y_train) # 2. 预测 y_pred = lr.predict(X_test_scaled) mse
= mean_squared_error(y_test, y_pred) rmse = np.sqrt(mse) r2 = r2_score(y_test,
y_pred) print(f"RMSE: {rmse:.4f}, R2: {r2:.4f}") # 3. 交叉验证 cv_scores =
cross_val_score(lr, X_train_scaled, y_train, cv=5, scoring='r2')
print(f"5-fold R2: {cv_scores.mean():.4f} (+/- {cv_scores.std():.4f})") # 4. 系数
coef_df = pd.DataFrame({'Feature': feature_names, 'Coef': lr.coef_})
print(coef_df.sort_values('Coef', key=abs, ascending=False))
```

注意事项

多重共线性

当特征间高度相关时 (如房屋面积和房间数)，OLS

估计的系数方差会很大，导致模型不稳定。解决方案包括删除冗余特征、使用 Ridge 正则化或 PCA 降维。

残差分析

线性回归假设残差服从正态分布且方差恒定（同方差性）。若残差图呈现漏斗形，说明存在异方差，需对目标变量取对数变换或使用加权回归。

常见问题

Q1： $R^2 = 0.85$ 表示什么？

A：表示模型解释了目标变量 85% 的方差。 R^2 越接近 1 越好，但可能受异常值影响。若 R^2 为负数，说明模型比简单预测均值还差。

Q2：特征标准化会影响线性回归的预测结果吗？

A：对于 OLS 求解，标准化不改变预测值，只改变系数的尺度和解释方式。但对于梯度下降，标准化能显著加速收敛。正则化线性回归（Ridge/Lasso）必须标准化，否则量纲大的特征会被过度惩罚。

实际案例

【案例】加州房价预测模型

使用 California Housing 数据集预测加州各区域房价中位数。数据集包含 8 个特征：收入中位数、房龄、房间数、卧室数、人口、家庭数、经纬度。

建模过程：1) 使用 StandardScaler 标准化；2) 训练 LinearRegression；3) 发现 R^2 为 0.606；4) 残差分析显示存在异方差，对目标取对数后 R^2 提升至 0.645；5) 加入经纬度的交互特征后， R^2 进一步提升至 0.682。

3.2 正则化线性回归：Ridge、Lasso 与 ElasticNet

正则化通过在损失函数中加入惩罚项来防止过拟合。Ridge (L2) 使权重趋于小但不为零；Lasso (L1) 可将不重要特征的系数压缩至零，实现特征选择；ElasticNet 结合两者优点。

知识点讲解

Ridge 回归损失函数： $J + \lambda \sum(\beta_j^2)$ 。L2 惩罚使系数平滑缩小，适合处理多重共线性。Lasso 回归损失函数： $J + \lambda \sum(|\beta_j|)$ 。L1 惩罚在坐标轴上产生角点，使部分系数恰好为零。

ElasticNet 结合 L1 和 L2： $\alpha \cdot L1_ratio \cdot |\beta| + 0.5 \cdot \alpha \cdot (1-L1_ratio) \cdot \beta^2$ 。当特征数远大于样本数或特征间高度相关时，ElasticNet 通常优于单独的 Lasso 或 Ridge。

操作步骤

步骤一：数据标准化。正则化对特征尺度敏感，必须先标准化。

步骤二：确定 α 搜索范围。从 10^{-4} 到 10^4 取对数尺度网格。

步骤三：使用 GridSearchCV 或 RidgeCV/LassoCV 进行交叉验证选择最优 alpha。

步骤四：对比不同模型的系数稀疏性和预测性能。

步骤五：对 Lasso 结果，保留非零系数对应的特征，实现特征选择。

```
from sklearn.linear_model import Ridge, Lasso, ElasticNet, RidgeCV from
sklearn.model_selection import GridSearchCV # Ridge ■■■L2■ ridge =
Ridge(alpha=1.0) ridge.fit(X_train_scaled, y_train) # Lasso ■■■L1■ lasso =
Lasso(alpha=0.1) lasso.fit(X_train_scaled, y_train) # ■■ RidgeCV ■■■■ alpha
ridge_cv = RidgeCV(alphas=np.logspace(-4, 4, 100), cv=5)
ridge_cv.fit(X_train_scaled, y_train) print(f"■■■ alpha: {ridge_cv.alpha_}") #
ElasticNet ■■■■ param_grid = { 'alpha': [0.001, 0.01, 0.1, 1, 10], 'l1_ratio':
[0.1, 0.3, 0.5, 0.7, 0.9] } grid = GridSearchCV(ElasticNet(max_iter=10000),
param_grid, cv=5, scoring='r2') grid.fit(X_train_scaled, y_train)
print(f"■■■■: {grid.best_params_}")
```

注意事项

alpha 参数含义

在 Ridge/Lasso 中，alpha 越大正则化越强，模型越简单。这与逻辑回归中的 C 参数相反（C 是正则化强度的倒数）。

Lasso 的稳定性

当特征间高度相关时，Lasso 倾向于随机选择其中一个特征，结果不稳定。此时应使用 ElasticNet 或先进行特征去相关。

常见问题

Q1：Ridge 和 Lasso 如何选择？

A：特征多且存在多重共线性 → Ridge；需要做特征选择、希望获得稀疏模型 → Lasso；特征高度相关且需要特征选择 → ElasticNet。

Q2：alpha=0 时 Ridge/Lasso 等价于什么？

A：alpha=0 时，正则化项消失，Ridge 和 Lasso 都退化为普通最小二乘线性回归（OLS）。

实际案例

【案例】基因表达数据的特征选择

某生物信息学项目需要从高维基因表达数据（20000+ 特征，200 样本）中筛选与癌症相关的关键基因。

1. 使用 Lasso 进行特征选择，交叉验证选择最优 alpha。
2. Lasso 将 20000+ 特征压缩至 47 个非零系数基因。
3. 使用这 47 个基因训练 SVM，在独立测试集上达到 92% 准确率。
4. 生物学验证确认其中 12 个基因与已知癌症通路相关。

第4章 分类算法：逻辑回归与朴素贝叶斯

4.1 逻辑回归原理与多分类策略

逻辑回归虽然名字含回归，但本质是分类算法。它通过 Sigmoid 函数将线性输出映射到 (0,1) 区间，表示样本属于正类的概率。

知识点讲解

Sigmoid 函数： $\sigma(z) = 1 / (1 + \exp(-z))$ 。当 $z \geq 0$ 时预测为正类， $z < 0$ 时预测为负类。损失函数为交叉熵损失（Cross-Entropy），衡量预测概率分布与真实分布的差异。

多分类策略：OvR（One-vs-Rest）训练 N 个二分类器，每个区分是某类 vs 其他；Softmax（Multinomial）直接输出各类概率，概率和为 1。Softmax 通常效果更优，但计算成本更高。

操作步骤

步骤一：数据标准化。逻辑回归对特征尺度敏感，需先标准化。

步骤二：选择 solver。小数据集用 liblinear（支持 L1），大数据集用 lbfgs 或 saga。

步骤三：设置 C 参数。C 是正则化强度的倒数，C 越大模型越复杂。

步骤四：处理类别不平衡。设置 class_weight='balanced' 自动调整权重。

步骤五：多分类设置。二分类默认即可，多分类设置 multi_class='multinomial'。

```
from sklearn.linear_model import LogisticRegression from sklearn.metrics import
(accuracy_score, precision_score, recall_score, f1_score, roc_auc_score,
classification_report) # 训练模型
model = LogisticRegression(C=1.0,
solver='lbfgs', max_iter=1000, class_weight='balanced', random_state=42)
model.fit(X_train_scaled, y_train) # 预测
y_pred = model.predict(X_test_scaled)
y_prob = model.predict_proba(X_test_scaled)[:, 1]
# 打印结果
print(f"Accuracy: {accuracy_score(y_test, y_pred):.4f}")
print(f"AUC: {roc_auc_score(y_test, y_prob):.4f}")
print(classification_report(y_test, y_pred))
# 多分类设置
multi_model = LogisticRegression(multi_class='multinomial',
solver='lbfgs', max_iter=1000)
multi_model.fit(X_train_scaled, y_train)
```

注意事项

max_iter 设置

当特征数多或数据未标准化时，优化可能不收敛，出现 ConvergenceWarning。应增加 max_iter（如 1000 或 5000），或检查数据是否标准化。

C 参数调优

C 参数通常在 0.001 到 100 之间搜索。C

过小会导致欠拟合，过大可能导致过拟合。建议在对数尺度上搜索。

常见问题

Q1：逻辑回归为什么不能用 MSE 作为损失函数？

A：分类问题中使用 MSE

会导致非凸优化，存在多个局部最优。交叉熵损失是凸函数，保证找到全局最优。

Q2：class_weight='balanced' 的原理是什么？

A：自动根据训练集中各类别频率计算权重，权重 = $n_samples / (n_classes * np.bincount(y))$ 。少数类获得更高权重，模型更关注少数类。

实际案例

【案例】乳腺癌良恶性分类

使用 Wisconsin Breast Cancer 数据集（30 个特征，569 个样本）进行二分类。

1. 数据标准化后训练 LogisticRegression(C=1.0, solver='lbfgs')。
2. 测试集准确率达 97%，AUC 达 0.995。
3. 通过系数绝对值识别最重要特征：worst perimeter、worst concave points、mean concave points。
4. 绘制 ROC 曲线，在 FPR=0.1 时 TPR 已达 0.98，说明模型在控制误诊率的同时保持了高检出率。

4.2 朴素贝叶斯分类器与文本分类应用

朴素贝叶斯基于贝叶斯定理和特征条件独立性假设。虽然独立性假设在现实中很少成立，但算法在文本分类等领域表现优异。

知识点讲解

三种变体：高斯朴素贝叶斯（GaussianNB）假设连续特征服从正态分布；多项式朴素贝叶斯（MultinomialNB）适合离散计数特征（如词频）；伯努利朴素贝叶斯（BernoulliNB）适合二值特征（如词是否出现）。

拉普拉斯平滑（alpha）：防止某个特征在某类别中未出现时概率为 0，导致整个后验概率为 0。alpha=1.0 是标准拉普拉斯平滑。

操作步骤

步骤一：特征分析。连续特征用 GaussianNB；文本词频用 MultinomialNB；二值特征用 BernoulliNB。

步骤二：文本预处理。分词、去停用词、TF-IDF 或 CountVectorizer 转换。

步骤三：训练模型。设置 alpha 进行平滑。

步骤四：评估。文本分类常用准确率、F1-Score、混淆矩阵。

步骤五：解释结果。通过 feature_log_prob_ 查看各类别关键词。

```
from sklearn.naive_bayes import MultinomialNB from
sklearn.feature_extraction.text import TfidfVectorizer # 1. 实例化 vectorizer =
TfidfVectorizer(stop_words='english', max_features=5000) X_train_vec =
vectorizer.fit_transform(train_texts) X_test_vec =
vectorizer.transform(test_texts) # 2. 训练模型 model =
MultinomialNB(alpha=1.0) model.fit(X_train_vec, y_train) # 3. 获取特征名称
feature_names = vectorizer.get_feature_names_out() for i, category in
enumerate(categories): top_idx = np.argsort(model.feature_log_prob_[i])[-10:]
top_words = [feature_names[j] for j in top_idx] print(f"{category}: {'',
'.join(reversed(top_words))}")
```

注意事项

特征独立性假设

朴素贝叶斯假设特征条件独立。若特征高度相关（如房屋面积和房间数），概率估计会有偏差。但分类决策只关心后验概率的相对大小，排序通常仍正确。

零概率问题

若不加平滑，测试集中出现训练时未见过的特征值会导致概率为 0。务必设置 $\alpha > 0$ ，通常 $\alpha=1.0$ 效果稳定。

常见问题

Q1：朴素贝叶斯为什么适合文本分类？

A：1) 训练速度极快，适合大规模数据；2) 对缺失值不敏感；3) 在高维稀疏数据（如词袋模型）上效果往往很好；4) 天然支持多分类。

Q2：GaussianNB 需要标准化吗？

A：理论上不需要，因为 GaussianNB 估计的是每个类别下每个特征的均值和方差，尺度不影响概率计算。但标准化后有时效果会略好。

实际案例

【案例】新闻文本自动分类系统

某内容平台需要自动将新闻分为 20 个类别（政治、科技、体育等）。

1. 使用 20 Newsgroups 数据集训练，去除邮件头和签名。

2. TF-IDF 特征提取 (max_features=5000, 去除英文停用词)。
3. MultinomialNB(alpha=1.0) 训练, 在 4 类子集上准确率达 84%。
4. 查看各类别关键词: sci.space 对应 space、orbit、launch; comp.graphics 对应 image、graphics、file。
5. 与 SVM 对比, NB 训练速度快 100 倍, 准确率仅低 3%, 在实时分类场景下性价比极高。

第5章 距离模型：KNN 与 SVM

5.1 KNN 算法原理与调优策略

KNN (K-Nearest Neighbors) 是一种惰性学习算法，没有显式的训练过程。核心思想是：一个样本的类别由其最近的 K 个邻居的多数类别决定。

知识点讲解

距离度量：欧氏距离（默认，适合连续特征）、曼哈顿距离（适合高维数据）、闵可夫斯基距离（ $p=1$ 为曼哈顿， $p=2$ 为欧氏）。权重策略：uniform（等权重投票）和 distance（按距离倒数加权，近邻权重更大）。

K 值选择：K 较小模型复杂，对噪声敏感，易过拟合；K 较大模型简单，决策边界平滑，易欠拟合。经验法则： $K = \sqrt{n}$ ，且通常取奇数避免平票。

操作步骤

步骤一：特征标准化。KNN 基于距离计算，必须标准化，否则量纲大的特征会主导距离。

步骤二：选择 K 值范围。通常从 1 到 $\sqrt{n}$ 搜索。

步骤三：交叉验证选择最优 K。使用 5 折或 10 折交叉验证。

步骤四：选择距离度量。默认欧氏距离，高维数据可尝试曼哈顿距离。

步骤五：选择权重策略。数据噪声大时用 distance 加权。

```
from sklearn.neighbors import KNeighborsClassifier from sklearn.model_selection
import cross_val_score, GridSearchCV # K 调优
k_range = range(1, 31) cv_scores = []
for k in k_range: knn = KNeighborsClassifier(n_neighbors=k) scores =
cross_val_score(knn, X_train_s, y_train, cv=5) cv_scores.append(scores.mean())
best_k = k_range[np.argmax(cv_scores)] # GridSearchCV 调优
param_grid = {
'n_neighbors': range(1, 21), 'weights': ['uniform', 'distance'], 'metric':
['euclidean', 'manhattan']} grid = GridSearchCV(KNeighborsClassifier(),
param_grid, cv=5) grid.fit(X_train_s, y_train) print(f"最佳 K 值: {grid.best_params_}")
```

注意事项

维度灾难

在高维空间中，所有样本之间的距离趋于相近，KNN 效果急剧下降。解决方案：先进行特征选择或降维（PCA），再使用 KNN。

预测效率

KNN 预测时需计算待预测样本与所有训练样本的距离，时间复杂度 $O(nd)$ 。大数据集下预测很慢，不适合实时性要求高的场景。

常见问题

Q1：为什么 KNN 必须进行特征标准化？

A：KNN 基于距离度量。若某特征量纲很大（如收入以元为单位，数值达数万），它会完全主导距离计算，掩盖其他特征的作用。标准化使各特征在同一尺度上公平竞争。

Q2：KNN 可以做回归吗？

A：可以。KNeighborsRegressor 取 K 个邻居目标值的平均（或加权平均）作为预测值。适用于房价预测等连续值预测任务。

实际案例

【案例】信用卡客户信用评分

某银行使用 KNN 对客户进行信用评分（A/B/C/D 四级）。

1. 特征包括：年龄、收入、负债比、历史逾期次数、信用卡使用年限。
2. 使用 StandardScaler 标准化后，交叉验证选择最优 $K=7$ 。
3. 对比 uniform 和 distance 权重，distance 权重在测试集上准确率提升 2%。
4. 由于客户数达百万级，KNN 预测过慢，最终仅用于小规模样本的辅助验证，主模型改用 XGBoost。

5.2 支持向量机 SVM 与核技巧

SVM 是一种最大化分类间隔的判别式模型。核心思想是找到一个最优超平面，使两类样本之间的间隔最大。间隔越大，泛化能力越强。

知识点讲解

线性可分 SVM：优化目标为 $\min 0.5 * ||w||^2$ ，约束条件为 $y_i * (w^T x_i + b) \geq 1$ 。支持向量是位于间隔边界上的样本点，决定了超平面位置。

软间隔与 C 参数：引入松弛变量和惩罚参数 C 。 C 很大时惩罚强，尽量正确分类所有点，易过拟合； C 很小时惩罚弱，允许更多误分类，易欠拟合。

核技巧：当数据线性不可分时，通过核函数将数据映射到高维空间。RBF 核（高斯核）是最通用的核函数， γ 控制单个样本的影响范围。

操作步骤

步骤一：数据标准化。SVM 基于距离计算，必须标准化。

步骤二：选择核函数。线性核优先尝试，不行再试 RBF。

步骤三：设置 C 和 gamma。C 从 0.1 到 100 搜索，gamma 从 scale/auto/0.001 到 1 搜索。

步骤四：使用 GridSearchCV 进行网格搜索。

步骤五：分析支持向量数量。支持向量过多说明模型复杂，可能过拟合。

```
from sklearn.svm import SVC from sklearn.model_selection import GridSearchCV #
RBF ■ SVM ■■■■ param_grid = { 'C': [0.1, 1, 10, 100], 'gamma': ['scale',
'auto', 0.001, 0.01, 0.1, 1] } grid = GridSearchCV(SVC(kernel='rbf'),
param_grid, cv=5, scoring='accuracy') grid.fit(X_train_s, y_train)
print(f"■■■■: {grid.best_params_}") print(f"■■■■■:
{grid.best_estimator_.n_support_}") # ■■ SVM■■■■■■■ from sklearn.svm import
LinearSVC linear_svm = LinearSVC(C=1.0, max_iter=10000)
linear_svm.fit(X_train_s, y_train)
```

注意事项

gamma 参数影响

gamma 越大，单个样本影响范围越小，决策边界越曲折，越容易过拟合。gamma 越小，影响范围越大，边界越平滑。默认 gamma='scale' 使用 $1/(n_features * X.var())$ 作为 gamma。

大数据集慎用

SVM 训练时间复杂度为 $O(n^2)$ 到 $O(n^3)$ 。当样本数超过 10 万时，训练时间可能无法接受。此时应使用 LinearSVC 或改用其他算法。

常见问题

Q1：SVM 中 C 参数与逻辑回归中 C 参数含义相同吗？

A：含义相同，都是正则化强度的倒数。C 越小正则化越强。但 SVM 的 C 直接控制对误分类的惩罚，而逻辑回归的 C 控制 L2 正则化强度。

Q2：为什么 SVM 的最终模型只依赖支持向量？

A：非支持向量满足 $y_i * (w^T x_i + b) > 1$ ，其拉格朗日乘子为 0，对模型参数 w 和 b 无贡献。因此删除非支持向量不影响模型。

实际案例

【案例】手写数字识别

使用 MNIST 数据集（70000 张 28x28 手写数字图片）进行分类。

1. 将图片展平为 784 维向量，标准化后使用 PCA 降至 50 维。
2. 使用 RBF 核 SVM，GridSearchCV 搜索最优 C=10, gamma=0.01。
3. 在测试集上准确率达 98.5%，与 CNN 的 99.2% 接近，但训练速度快于深度学习。

4. 分析发现数字 3 和 5 最容易混淆，支持向量主要集中在这些困难样本上。

第6章 决策树与集成学习基础

6.1 决策树原理与剪枝策略

决策树是一种树形结构的分类/回归模型，通过一系列规则对数据进行判断。优势在于可解释性强，无需特征标准化，能处理非线性关系。

知识点讲解

分裂准则：CART 分类树默认使用基尼指数（Gini），也可选择信息增益（Entropy）。基尼指数衡量从数据集中随机抽取两个样本，其类别不一致的概率。信息增益基于熵的概念，熵衡量系统的不确定性。

防止过拟合的方法：预剪枝（在构建树时限制复杂度，参数包括 `max_depth`、`min_samples_split`、`min_samples_leaf`）和后剪枝（先构建完整树，再通过 `ccp_alpha` 代价复杂度剪枝）。

操作步骤

步骤一：训练完整决策树。不限制参数，观察是否过拟合。

步骤二：预剪枝调参。从 `max_depth=3` 开始逐步增加，观察训练集和测试集性能差距。

步骤三：使用后剪枝。调用 `cost_complexity_pruning_path` 获取剪枝路径，选择使测试集性能最优的 `ccp_alpha`。

步骤四：可视化树结构。使用 `plot_tree` 或 `export_text` 查看决策规则。

步骤五：分析特征重要性。基于不纯度减少计算，可用于特征选择。

```
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text from
sklearn.model_selection import GridSearchCV # 1. 定义参数网格 param_grid = {
    'max_depth': [2, 3, 5, 7, 10, None], 'min_samples_split': [2, 5, 10, 20],
    'min_samples_leaf': [1, 2, 4, 8], 'criterion': ['gini', 'entropy']} grid =
GridSearchCV(DecisionTreeClassifier(random_state=42), param_grid, cv=5,
scoring='accuracy') grid.fit(X_train, y_train) # 2. 获取最佳模型 dt =
grid.best_estimator_ # 3. 计算代价复杂度路径 path =
dt.cost_complexity_pruning_path(X_train, y_train) ccp_alphas = path.ccp_alphas
# 4. 选择最优的 ccp_alpha # 5. 可视化决策树 plt.figure(figsize=(20, 12)) plot_tree(dt,
feature_names=feature_names, class_names=class_names, filled=True,
rounded=True, fontsize=10) plt.savefig('decision_tree.png', dpi=150,
bbox_inches='tight')
```

注意事项

不稳定性

决策树对数据的微小变化敏感，可能导致完全不同的树结构。解决方案是使用集成方法（随机森林）来降低方差。

偏向性

决策树倾向于选择取值较多的特征（如用户ID）进行分裂。应对方法是限制 `max_features` 或使用信息增益率替代信息增益。

常见问题

Q1：基尼指数和熵哪个更好？

A：实际效果通常相近。基尼指数计算更快（无需计算对数），熵在数学上更严谨。sklearn 默认使用基尼指数，但两者差异通常小于 1%。

Q2：`max_depth` 设为 `None` 会怎样？

A：树会生长到每个叶节点都是纯的（所有样本同属一类）或 `min_samples_split` 限制为止。这通常会导致严重过拟合，训练集准确率 100% 但测试集很差。

实际案例

【案例】贷款审批决策系统

某银行需要构建可解释的贷款审批模型，供风控人员审核使用。

1. 使用决策树，限制 `max_depth=5`，确保规则简单易懂。
2. 关键规则：年收入 < 5万 且 信用评分 < 600 → 拒绝；年收入 > 20万 且 无逾期记录 → 通过。
3. 风控人员可直接查看决策路径，理解模型拒绝某笔贷款的原因。
4. 虽然准确率（85%）低于 XGBoost（92%），但可解释性满足监管要求。

6.2 Bagging 与随机森林

Bagging (Bootstrap Aggregating) 通过并行训练多个同质模型，对预测结果取平均（回归）或投票（分类）来降低方差。随机森林是 Bagging 的增强版，在采样样本的同时还随机选择特征子集。

知识点讲解

Bagging 核心：从原始数据中有放回地抽取 n 个样本构建训练子集，训练多个基学习器。预测时分类任务用多数投票，回归任务用平均。

随机森林改进：每棵树分裂时，随机选择 `max_features` 个特征参与分裂。默认 `max_features='sqrt'`（分类）或 `'log2'`（回归）。这进一步增加了树之间的多样性。

Out-of-Bag (OOB) 评估：约 37% 的样本不会被某棵树采样到，可用作该树的验证集。设置 `oob_score=True` 即可自动计算 OOB 分数，无需额外划分验证集。

操作步骤

步骤一：确定 `n_estimators`。通常越大越好，但收益递减。100~500 是常用范围。

步骤二：设置 `max_depth`。防止单棵树过深，通常 10~30。

步骤三：设置 `max_features`。分类问题默认 `sqrt`，可尝试 `log2` 或具体数值。

步骤四：使用 OOB 分数快速评估。无需交叉验证即可估计泛化性能。

步骤五：分析特征重要性。基于不纯度减少的加权平均。

```
from sklearn.ensemble import RandomForestClassifier # ■■■■ rf =
RandomForestClassifier( n_estimators=200, max_depth=15, max_features='sqrt',
min_samples_split=5, oob_score=True, n_jobs=-1, random_state=42 )
rf.fit(X_train, y_train) print(f"OOB Score: {rf.oob_score_:.4f}") print(f"Test
Accuracy: {rf.score(X_test, y_test):.4f}") # ■■■■ importance = pd.DataFrame({
'Feature': feature_names, 'Importance': rf.feature_importances_
}).sort_values('Importance', ascending=False) print(importance.head(10))
```

注意事项

`n_estimators` 与训练时间

`n_estimators` 增加会降低方差，但训练时间线性增长。当 OOB 分数不再显著提升时，即可停止增加。通常 200~500 棵树已足够。

特征重要性偏差

随机森林的特征重要性对高基数类别特征（如用户ID）有偏向。若需无偏估计，应使用排列重要性（`permutation_importance`）。

常见问题

Q1：随机森林为什么比单棵决策树更稳定？

A：通过 Bagging 平均多棵树的预测，随机森林的方差大幅降低。单棵决策树对数据微小变化敏感，而森林中部分树的变化被其他树抵消。

Q2：`max_features='sqrt'` 是什么意思？

A：每次分裂时，随机选择 `sqrt(n_features)` 个特征参与最优分裂点的搜索。例如 100 个特征时，每次只考虑 10 个特征，增加树之间的多样性。

实际案例

【案例】电信客户流失预测

某电信运营商使用随机森林预测客户是否会流失。

- 数据集包含 7000 条客户记录，20 个特征（合约类型、月消费、客服投诉次数等）。
- 随机森林（`n_estimators=200`, `max_depth=10`）在测试集上准确率达 91%，AUC 达 0.94。

3. 特征重要性分析：合约剩余月份（0.28）、月消费金额（0.19）、客服投诉次数（0.15）为前三重要特征。

4. 业务洞察：合约即将到期的客户流失风险最高，运营团队据此推出续约优惠，3个月内流失率下降8%。

第7章 Boosting 算法：AdaBoost 与 GBDT

7.1 AdaBoost 原理与实现

AdaBoost (Adaptive Boosting) 是一种串行训练的集成方法。每个新基学习器重点关注前一个模型分错的样本，通过调整样本权重逐步提升整体性能。

知识点讲解

核心机制：初始化样本权重为均匀分布。每轮迭代中，根据当前权重训练基学习器，计算错误率 ϵ_t 和学习器权重 $\alpha_t = 0.5 * \ln((1-\epsilon_t)/\epsilon_t)$ 。然后更新样本权重：分错的样本权重增加，分对的样本权重减少。

基学习器选择：通常使用决策树桩 (Decision Stump，即 $\text{max_depth}=1$ 的决策树) 作为基学习器。树桩简单但多样性好，适合 Boosting 框架。

sklearn 中两种算法：SAMME (离散版本，使用类别预测) 和 SAMME.R (实数版本，使用概率估计)。SAMME.R 收敛更快，是默认选项。

操作步骤

步骤一：选择基学习器。通常使用 `DecisionTreeClassifier(max_depth=1)`。

步骤二：设置 `n_estimators`。迭代轮数，通常 50~500。

步骤三：设置 `learning_rate`。学习率与 `n_estimators` 存在权衡：学习率小则需要更多树。

步骤四：训练模型并监控 `staged_predict` 的学习曲线。

步骤五：分析 `estimator_errors_` 和 `estimator_weights_` 了解每轮表现。

```
from sklearn.ensemble import AdaBoostClassifier from sklearn.tree import
DecisionTreeClassifier # AdaBoost ■■■ ada = AdaBoostClassifier(
estimator=DecisionTreeClassifier(max_depth=1), n_estimators=100,
learning_rate=0.1, algorithm='SAMME.R', random_state=42 ) ada.fit(X_train,
y_train) # ■■■■■■■■■■■■ for i, (err, weight) in
enumerate(zip(ada.estimator_errors_[1:], ada.estimator_weights_[1:])):
print(f"Round {i+1}: error={err:.4f}, weight={weight:.4f}") # staged_predict
■■■■■ from sklearn.metrics import accuracy_score for i, pred in
enumerate(ada.staged_predict(X_test)): if i % 20 == 0: print(f"Iteration {i+1}:
{accuracy_score(y_test, pred):.4f}")
```

注意事项

对噪声敏感

AdaBoost 会不断关注难分样本，若数据中存在噪声（标签错误的样本），AdaBoost 会持续增加这些样本的权重，导致过拟合。数据清洗对 AdaBoost 尤为重要。

学习率与迭代次数

learning_rate 和 n_estimators 存在 trade-off：learning_rate=1.0 时 n_estimators 不宜过大；learning_rate=0.01 时可能需要 1000+ 棵树。常用组合：learning_rate=0.1, n_estimators=100。

常见问题

Q1：AdaBoost 中如果某基学习器错误率大于 0.5 会怎样？

A：当 $\epsilon_t > 0.5$ 时， α_t 为负数，意味着该学习器比随机猜测还差。此时算法应停止或重新采样。实际中可通过限制基学习器复杂度来避免。

Q2：AdaBoost 的基学习器可以是任意模型吗？

A：理论上可以，但实际中通常使用决策树桩。因为 Boosting 需要基学习器支持样本权重（sample_weight 参数），且弱学习器（高偏差低方差）在 Boosting 框架下提升最明显。

实际案例

【案例】人脸识别中的 AdaBoost

Viola-Jones 人脸检测算法是 AdaBoost 的经典应用，被用于早期数码相机的人脸对焦功能。

1. 特征：Haar 特征（边缘、线、中心环绕特征），每幅图可提取数万维特征。
2. 使用 AdaBoost 选择最重要的数百个 Haar 特征，每个特征对应一个弱分类器（决策树桩）。
3. 级联结构：先用简单分类器快速排除非人脸区域，再用复杂分类器精细判断。
4. 在 384x288 图像上检测速度达 15 FPS，误检率低于 1/100000。

7.2 GBDT 梯度提升决策树

GBDT (Gradient Boosting Decision Tree) 是 Boosting 的进阶版本。与 AdaBoost 通过调整样本权重来关注错误样本不同，GBDT 通过拟合前一轮模型的残差（负梯度）来逐步减小损失。

知识点讲解

核心思想：第 m 棵树拟合的是前 $m-1$ 棵树的预测值与真实值之间的残差（对于平方损失）或负梯度（对于一般损失函数）。最终预测是所有树预测值的加权和。

关键超参数：n_estimators（树的数量）、learning_rate（每棵树的贡献权重，通常 0.01~0.1）、max_depth（单棵树深度，通常 3~5）、subsample（子采样比例，用于随机梯度提升）。

损失函数选择：分类任务用对数损失（deviance），回归任务用均方误差（squared_error）或绝对误差（absolute_error）。

操作步骤

步骤一：设置损失函数。分类用 'log_loss'，回归用 'squared_error'。

步骤二：设置 max_depth。通常 3~5，防止单棵树过拟合。

步骤三：设置 learning_rate 和 n_estimators。常用组合：(0.1, 100)、(0.05, 200)、(0.01, 500)。

步骤四：使用早停（validation_fraction + n_iter_no_change）防止过拟合。

步骤五：分析特征重要性，识别关键预测因子。

```
from sklearn.ensemble import GradientBoostingClassifier # GBDT
gbdt = GradientBoostingClassifier( n_estimators=200, learning_rate=0.1, max_depth=3,
min_samples_split=5, subsample=0.8, # validation_fraction=0.1,
n_iter_no_change=10, # random_state=42 )
gbdt.fit(X_train, y_train)
print(f"GBDT: {gbdt.n_estimators}")
print(f"OOB Score: {gbdt.train_score}") #
importance = pd.DataFrame( { 'Feature': feature_names, 'Importance':
gbdt.feature_importances_ } ).sort_values('Importance', ascending=False)
```

注意事项

过拟合风险

GBDT 容易过拟合，尤其是 n_estimators 过大或 max_depth 过深时。务必使用验证集监控性能，或使用早停策略。

训练速度

sklearn 的 GradientBoostingClassifier 训练较慢。大规模数据建议使用 XGBoost、LightGBM 或 sklearn 的 HistGradientBoostingClassifier（基于直方图，速度更快）。

常见问题

Q1：GBDT 与 AdaBoost 的核心区别是什么？

A：AdaBoost 通过调整样本权重来关注错误样本；GBDT 通过拟合残差（负梯度）来逐步减小损失。GBDT 可以使用任意可微损失函数，更灵活。

Q2：为什么 GBDT 的树深度通常设为 3~5？

A：GBDT 中每棵树只需拟合残差，不需要很强的拟合能力。浅树（高偏差低方差）在 Boosting 框架下通过多轮迭代逐步提升，比深树更稳定、更不易过拟合。

实际案例

【案例】房价预测中的 GBDT

Kaggle 房价预测竞赛中，GBDT 是主流方案之一。

1. 特征工程：房屋面积、卧室数、地段评分、建造年份等 80+ 特征。
2. GBDT 参数：n_estimators=500, learning_rate=0.05, max_depth=4, subsample=0.8。
3. 使用对数变换处理右偏的目标变量（房价），使残差更接近正态分布。
4. 交叉验证 RMSLE（对数均方根误差）为 0.12，排名进入前 10%。

第8章 XGBoost 与模型评估部署

8.1 XGBoost 原理与实战调优

XGBoost (eXtreme Gradient Boosting) 是 GBDT 的高效实现，在各类数据竞赛中表现优异。它在 GBDT 基础上加入了正则化、缺失值处理、并行计算等优化。

知识点讲解

XGBoost 的核心优化包括：1) 正则化项 (L1 和 L2) 防止过拟合；2) 二阶泰勒展开近似损失函数，收敛更快；3) 缺失值自动处理（学习最优分裂方向）；4) 列采样和行采样增加随机性；5) 近似算法和加权分位数草图加速分裂点搜索。

关键参数：max_depth（树深度，3~6）、learning_rate（学习率，0.01~0.3）、n_estimators（树数量，用早停自动确定）、subsample（行采样比例）、colsample_bytree（列采样比例）、reg_alpha（L1 正则化）、reg_lambda（L2 正则化）。

操作步骤

步骤一：安装 XGBoost。 `pip install xgboost`

步骤二：设置基础参数。max_depth=3~6, learning_rate=0.1, objective 根据任务选择。

步骤三：设置采样参数。subsample=0.8, colsample_bytree=0.8 增加随机性。

步骤四：使用 `eval_set` 和 `early_stopping_rounds` 自动确定最优迭代次数。

步骤五：使用 GridSearchCV 或 Optuna 进行超参数调优。

```
import xgboost as xgb from sklearn.model_selection import train_test_split #
#####/##### X_tr, X_val, y_tr, y_val = train_test_split(X_train, y_train,
test_size=0.2) # XGBoost ### model = xgb.XGBClassifier( max_depth=5,
learning_rate=0.1, n_estimators=1000, subsample=0.8, colsample_bytree=0.8,
reg_alpha=0.1, reg_lambda=1.0, objective='binary:logistic', eval_metric='auc',
use_label_encoder=False, random_state=42 ) # ##### model.fit( X_tr, y_tr,
eval_set=[(X_val, y_val)], early_stopping_rounds=50, verbose=False )
print(f"#####: {model.best_iteration}") print(f"#### AUC:
{model.best_score:.4f}") # #####weight, gain, cover#
xgb.plot_importance(model, importance_type='gain', max_num_features=10)
```

注意事项

类别编码

XGBoost 不能直接处理字符串类别特征。需先用 LabelEncoder 或 OrdinalEncoder 编码。最新版本支持 enable_categorical=True 直接处理类别特征。

类别不平衡

对于类别不平衡问题，设置 $\text{scale_pos_weight} = \text{sum(negative)} / \text{sum(positive)}$ 。这相当于给正类样本更高的权重，等效于过采样。

常见问题

Q1：XGBoost 的 early_stopping_rounds 是什么意思？

A：如果在连续 early_stopping_rounds 轮内，验证集性能没有提升，则停止训练。这自动确定了最优 n_estimators，防止过拟合。

Q2：XGBoost 的 gain 和 weight 重要性有什么区别？

A：weight 统计特征被用于分裂的次数；gain 统计特征带来的平均损失减少量。gain 更能反映特征的真实重要性，weight 可能偏向高频特征。

实际案例

【案例】Kaggle 泰坦尼克号生存预测

泰坦尼克号是 Kaggle 最经典的入门竞赛。

1. 特征工程：姓名提取头衔 (Mr/Mrs/Miss)、家庭大小 (SibSp+Parch+1)、票价分箱、登船港口编码。
2. XGBoost 参数：max_depth=4, learning_rate=0.05, n_estimators=500, subsample=0.9。
3. 使用 5 折交叉验证，平均准确率达 83.5%，排名前 8%。
4. SHAP 分析发现：性别 (female 生存率远高于 male)、票价等级 (Pclass=1 生存率高)、年龄 (儿童优先) 是最重要的预测因子。

8.2 模型评估、选择与生产部署

模型训练完成后，需要全面评估其性能、选择最优模型，并将其部署到生产环境。这是机器学习工程师的核心职责之一。

知识点讲解

评估指标选择：分类任务中，类别平衡用准确率；类别不平衡用 F1-Score、AUC-ROC、AUC-PR；关注误报用精确率；关注漏报用召回率。回归任务中，RMSE 与目标同量纲最直观；MAE 对异常值鲁棒；R² 解释方差比例。

交叉验证策略：K 折交叉验证（K-Fold）适用于数据量中等；分层 K 折（StratifiedKFold）适用于类别不平衡；时间序列分割（TimeSeriesSplit）适用于时序数据，避免未来信息泄漏。

模型部署方式：批处理（Batch Prediction，每日/每小时批量预测）、实时 API（Flask/FastAPI 封装 REST API）、模型服务化（MLflow、BentoML、Triton Inference Server）。

操作步骤

步骤一：选择评估指标。根据业务场景确定主指标和辅助指标。

步骤二：交叉验证评估。使用 StratifiedKFold 进行 5 折或 10 折验证。

步骤三：模型对比。训练多个候选模型，用交叉验证结果选择最优模型。

步骤四：超参数调优。使用 GridSearchCV、RandomizedSearchCV 或 Optuna。

步骤五：模型持久化。使用 joblib 或 pickle 保存训练好的模型。

步骤六：部署上线。封装为 API，设置监控和告警。

```
from sklearn.model_selection import StratifiedKFold, cross_val_score import
joblib # ■■■ K ■■■■■ skf = StratifiedKFold(n_splits=5, shuffle=True,
random_state=42) cv_scores = cross_val_score(model, X, y, cv=skf,
scoring='roc_auc') print(f"5■ AUC: {cv_scores.mean():.4f} (+/-
{cv_scores.std():.4f})") # ■■■■■ joblib.dump(model, 'model.pkl')
joblib.dump(preprocessor, 'preprocessor.pkl') # ■■■■■ loaded_model =
joblib.load('model.pkl') loaded_preprocessor = joblib.load('preprocessor.pkl')
X_new_processed = loaded_preprocessor.transform(X_new) predictions =
loaded_model.predict(X_new_processed)
```

注意事项

训练-测试分布偏移

生产环境中，数据分布可能随时间变化（概念漂移）。需定期监控模型性能，当性能下降超过阈值时触发重训练。

模型版本管理

生产模型应记录训练时间、数据版本、超参数、评估指标。使用 MLflow 或 Weights & Biases 进行实验追踪和模型版本管理。

常见问题

Q1：为什么时间序列数据不能用普通 K 折交叉验证？

A：普通 K 折随机打乱数据，可能导致训练集中包含测试集的未来信息，造成泄漏。时间序列应使用 TimeSeriesSplit，确保训练集始终早于测试集。

Q2：AUC-ROC 和 AUC-PR 有什么区别？

A：AUC-ROC 在所有阈值下综合评估 TPR 和 FPR；AUC-PR 在正类稀少时更敏感，因为它关注精确率-召回率曲线下的面积。类别极度不平衡时，AUC-PR 比

AUC-ROC 更有参考价值。

实际案例

【案例】电商推荐系统的模型部署与监控

某电商平台构建商品点击率预测模型，用于个性化推荐。

1. 模型选择：对比逻辑回归（可解释）、随机森林（稳定）、XGBoost（精度高），最终选择 XGBoost。
2. 部署方式：使用 FastAPI 封装为 REST API，部署在 Kubernetes 集群，支持水平扩展。
3. 监控指标：每秒请求数（QPS）、平均响应时间（P99 < 50ms）、预测分布漂移（KL 散度）。
4. 当 AUC 连续 3 天下降超过 0.02 时，自动触发重训练流水线，使用最新 7 天数据重新训练并 A/B 测试后上线。